

What is Terraform?

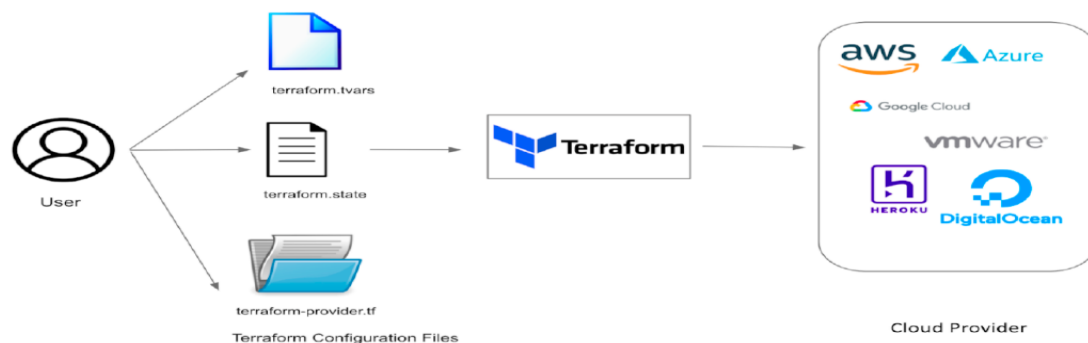
- Terraform is an **infrastructure as a code** tool that lets you build, change and version control cloud resources safely, easily and efficiently
- Terraform is an open source "Infrastructure as a Code" tool, created by HashiCorp.
- Terraform is developed using Go Language in the year 2014.
- All the configuration files used HCL (HashiCorp Configuration Language) language for the code.
- Terraform uses a simple syntax, can provision infrastructure across multiple clouds & On premises.
- Terraform is cloud-agnostic and supports multiple cloud providers such as AWS, Azure, Google Cloud, and others

Infrastructure as Code(IaC)

Before the invention of IaC, infrastructure management was typically a manual and time-consuming process.

System administrators and operations teams had to:

1. Manually Configure Servers
2. Lack of Version Control
3. Documentation Heavy
4. Limited Automation
5. Slow Provisioning



Alternatives Of Terraform:

CLOUD	IAC tool (Native)	Configuration written in
AWS	CFT (cloud formation template)	JSON / YAML
AZURE	ARM TEMPLATES	JSON
GCP	CLOUD DEPLOYMENT MANAGER	YAML / PYTHON

Features of Terraform:

Terraform is a tool that helps you set up and manage cloud infrastructure using code.

It lets you define how the infrastructure should be, and Terraform will take care of setting it up.

Multi-Cloud Support:

- Terraform can work with different cloud platforms like AWS, Azure, Google Cloud, and others.
- It can also be used with on-premise servers or a mix of both.

Declarative Configuration:

In simple terms, you write down how you want your infrastructure to look in a configuration file (using a special language called HCL). and Terraform takes care of figuring out how to achieve that state.

Resource Provisioning:

- Terraform helps you create and manage things like virtual machines, storage, and networks and more.
- It creates and updates resources based on the configuration provided.

State Management:

- Terraform maintains a state file that keeps track of the current state of the infrastructure.
- This file is used to plan and apply changes, ensuring that Terraform can update resources accurately.

Plan and Apply Workflow:

- Before changing anything, Terraform creates a plan showing what it will do.
- You can review this plan and then apply it to make the changes.

Version Control Integration:

- You can store your Terraform configuration files (except state files) in version control tools like Git.
- This helps you work together with others, check for changes, and keep track of updates.

Modular Configuration:

- You can organize your infrastructure setup into modules.
- modules make easier to reuse and share configuration files across different projects.

Community support:

Terraform has a big community and many ready-made solutions for common infrastructure needs, which makes it easier for you to get started.

TERRAFORM LIFECYCLE:

The Terraform lifecycle refers to the sequence of steps and processes that occur when working with Terraform to manage infrastructure as code.



Write your infrastructure in Configuration file.

Users define their infrastructure in a declarative configuration language, commonly using HashiCorp Configuration Language (HCL).

Initialize (**terraform init**):

terraform init command will initialize a Terraform working directory.

This command initializes the Terraform working directory & also downloads provider plugins.

Plan (**terraform plan**):

terraform plan command creates an execution plan.

Terraform compares the desired state from the configuration with the current state and generates a plan for the changes required to reach the desired state.

Review Plan: Examine the output of the plan to understand what changes Terraform intends to make to the infrastructure. This is an opportunity to verify the planned changes before applying them.

Apply (**terraform apply**):

Execute terraform apply command to applies the changes outlined in the plan.

Terraform makes the necessary API calls to create, update, or delete resources to align the infrastructure with the desired state

Destroy(**terraform destroy**) :

When infrastructure is no longer needed, or for testing purposes, run terraform destroy to tear down all resources created by Terraform. This is irreversible, so use with caution.

Create an AWS IAM User:

To interact with AWS programmatically, you should create an IAM (Identity and Access Management) user with appropriate permissions.

Steps:

- Log in to the AWS Management Console with an account that has administrative privileges.
- Navigate to the IAM service.
- Click on “Users” in the left navigation pane and then click “Add user.”

Give username like **tf-user-1**

Specify user details

User details

User name: test1

☒ Provide user access to the AWS Management Console - optional

Are you providing console access to a person?

☒ Specify a user in Identity Center - Recommended

☐ I want to create an IAM user

Console password

☐ Auto-generated password

☒ Custom password: kewm9h138

☐ Show password

☐ If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.

Attach policy to user – add **AdministratorAccess** policy

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☒ Add user to group

☐ Copy permissions

☐ Attach policies directly

Permissions policies (1/1337)

Choose one or more policies to attach to your new user.

Policy name	Type	Attached entities
abhi-eks-ELNFK2-cluster-ClusterEncryption20250...	Customer managed	1
AccessAnalyzerServiceRolePolicy	AWS managed	0
AdministratorAccess	AWS managed - job function	4
AdministratorAccess-Amydify	AWS managed	0
AdministratorAccess-AWS'Elasticbeanstalk	AWS managed	0
AIOpsAssistantPolicy	AWS managed	0
AIOpsConsoleAdminPolicy	AWS managed	0

Click on view user

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

[View user](#)

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Console sign-in URL: https://242201285197.signin.aws.amazon.com/console

User name: test1

Console password: kewm9h138

[Download .csv file](#) [Return to users list](#)

Create an access keys for user

test1 [info](#) [Delete](#)

Summary

ARN: [arn:aws:iam::242201285197:user/test1](#)

Created: March 17, 2025, 08:57 (UTC+05:30)

Console access: [Enabled without MFA](#)

Last console sign-in: [Never](#)

Access key 1: [Create access key](#)

Permissions Groups Tags **Security credentials** Last Accessed

Console sign-in [Manage console access](#)

Console sign-in link: [https://d42201285197.signin.aws.amazon.com/console](#)

Console password: Updated 8 minutes ago (2025-03-17 08:57 GMT+5:30)

Last console sign-in: [Never](#)

Multi-factor authentication (MFA) (0) [Remove](#) [Resync](#) [Assign MFA device](#)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type	Identifier	Certifications	Created on
No MFA devices. Assign an MFA device to improve the security of your AWS environment.			

[Assign MFA device](#)

Access keys (0) [Create access key](#)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. [Learn more](#)

[Create access key](#)

SSH public keys for AWS CodeCommit (0) [Actions](#) [Upload SSH public key](#)

User SSH public keys to authenticate access to AWS CodeCommit repositories. You can have a maximum of five SSH public keys (active or inactive) at a time. [Learn more](#)

SSH Key ID	Uploaded	Status
------------	----------	--------

Step 1: Access key best practices & alternatives

Step 2 - optional: Set description tag

Step 3: Retrieve access keys

Access key best practices & alternatives [info](#)

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

- ☒ **Command Line Interface (CLI)**
You plan to use this access key to enable the AWS CLI to access your AWS account.
- ☐ **Local code**
You plan to use this access key to enable application code in a local development environment to access your AWS account.
- ☐ **Application running on an AWS compute service**
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.
- ☐ **Third-party service**
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.
- ☐ **Application running outside AWS**
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.
- ☐ **Other**
Your use case is not listed here.

Alternatives recommended

- Use [AWS CloudShell](#), a browser-based CLI, to run commands. [Learn more](#)
- Use the [AWS CLI V2](#) and enable authentication through a user in IAM Identity Center. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

[Cancel](#) [Next](#)

Add tag for reference

Step 1: Access key best practices & alternatives

Step 2 - optional: **Set description tag**

Step 3: Retrieve access keys

Set description tag - optional [info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

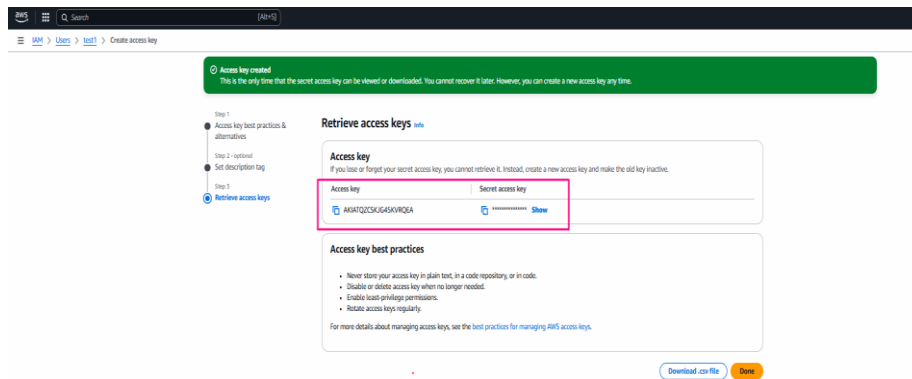
[Learn more](#) of this access key and where it will be used. A good description will help you locate this access key confidently later.

test-user-key

Maximum 256 characters. Allowed characters are letters, numbers, spaces, hyphens, and underscores. Do not use spaces at the beginning or end.

[Cancel](#) [Previous](#) [Create access key](#)

Terraform Class 1



Make sure to **save the Access Key ID and Secret Access Key** that are displayed after user creation;

Install Terraform:

Installation steps - <https://developer.hashicorp.com/terraform/install>

Launch ec2instance (t2.micro) with amazonlinux flavour & install terraform from below steps

```
sudo yum install -y yum-utils shadow-utils
```

```
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

```
sudo yum -y install terraform
```

verify installation:

```
[ec2-user@ip-172-31-26-175 ~]$ terraform -version
Terraform v1.11.2
on linux_amd64
[ec2-user@ip-172-31-26-175 ~]$
```

Note: Install **terraform** plugin in visual studio

Ex1. Write an terraform file to Create an VM in aws cloud (ec2 instance)

Create a directory called ec2instance & inside that create a .tf file

```
provider "aws" {
    # in this block we will specify which cloud we are using, which region &
    # credentials
    region = "us-east-1"
    access_key = "AKIAUEBQJSGXSDQ6GPFQ"
    secret_key = "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"
}

resource "aws_instance" "my-instance-1" {
    ami = "ami-04b4f1a9cf54c11d0"
    instance_type = "t2.micro"
    key_name = "terraform-kp-1"
}
```

Practical's & observation:

Step1: after creating file in directory run ***terraform init*** to initialize terraform & terraform also download AWS plugin file

Step2: run ***terraform validate*** command, it will check if the syntax is valid or any changes required.

Step3: run ***terraform plan*** command, check for the output for plan

Step4: run ***terraform apply*** command, which will create the resource (ec2 instance) specified in the .tf files

Step5: run ***terraform destroy*** command, which will delete all the resources created from that .tf file

Explanation of above Terraform script is used to provision an AWS EC2 instance.

Provider Block:

```
provider "aws" {  
    region = "us-east-1"           # Specifies the AWS region where resources will be created.  
    access_key = "AKIAUEBQGJ5OXXDQ6GTMG" # AWS access key for authentication (this key is specific to your AWS account).  
    secret_key = "a/v7k9ZAsZasQKH3WNo4s95/0sB5ZCNr10Ti/Kj1" # AWS secret key for authentication (this key is specific  
to your AWS account).  
}
```

- **Provider:** This tells Terraform which cloud platform you're using. In this case, it's AWS (Amazon Web Services).
- **Region:** It specifies that the resources should be created in the us-east-1 region of AWS.
- **Access Key** and **Secret Key:** These are the credentials used by Terraform to authenticate and interact with AWS. In practice, **you should avoid hardcoding keys** in the script for security reasons.

Resource Block:

```
resource "aws_instance" "my-instance-1" {  
    ami = "ami-04b4f1a9cf54c11d0" # The Amazon Machine Image (AMI) ID used to create the instance.  
    instance_type = "t2.micro"      # Specifies the type of instance (in this case, a small instance).  
    key_name = "terraform-kp-1"    # The name of the key pair to access the instance once it's running.  
}
```

- **Resource:** This block defines a resource that will be created. Here, it's an **EC2 instance** (aws_instance).
- **AMI:** The AMI ID ami-04b4f1a9cf54c11d0 specifies which operating system and software setup will be used for the instance.

- **Instance Type:** t2.micro is a small & free instance type on AWS.
- **Key Name:** The key pair (terraform-kp-1) is used to securely access the EC2 instance via SSH after it is created.

In summary:

- This Terraform script is setting up an EC2 instance in AWS using a specific AMI (operating system image), instance type (t2.micro), and key pair (terraform-kp-1) for SSH access.
- The provider block tells Terraform to interact with AWS and specifies the credentials and region to be used.

Important Note:

For security reasons, it's highly recommended **not to hardcode your access and secret keys** directly in your Terraform script. Instead, use environment variables, AWS IAM roles, or other secure methods to provide credentials.